CMPT 225 PROJECT FALL 2025

Dou Gwon
301620005 | dga73@sfu.ca

BFS	DFS
Finds the shortest distance from the starting vertex to all v∈V.	Usually does not give the shortest distance.
Efficient if the target is close to the starting vertex.	Reaches far away vertices quicker.
More structured	More useful for solving puzzles than BFS.
	Useful for other applications (topological search)

Source from CMPT 225 Week9 slide

1. **BFS (Not selected)**

The theoretical BFS time complexity O(|V|+|E|) applies only when the entire graph is explicitly stored as an adjacency list or adjacency matrix. When we use BFS for the Rubik's cube, its state-space graph contains approximately 4.3×10^19 states, which is too large to store explicitly in memory. So, an implicit graph representation must be used. In this case, as Korf (2008) demonstrated in his seminal paper, b (branching factor) of approximately 12–18 and d (solution depth) around 20.

e.g.

Depth 0: 1 node (start)
Depth 1: 12 nodes (12^1)
Depth 2: 144 nodes (12^2)
Depth 3: 1,728 nodes (12^3)
...
Depth d: b^d nodes → This produces exponential growth

O(b^d) is computationally impossible and can lead to memory exhaustion. For these reasons, BFS is not a feasible algorithm for the solution. However, when movements are not that big, we can use BFS. I used BFS for validation of RubiksCube file. It worked only for scramble01 and 02. Txt.

Reference

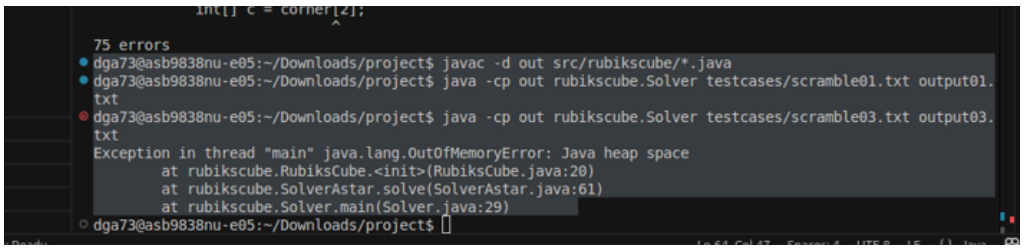
Korf, R. E. (2008). Rubik's Cube: Analyzing the complexity of optimal solutions.

2. **DFS (Not selected)**

Although DFS is sometimes mentioned as being useful for certain types of puzzles, this typically applies only to small problems with a low branching factor. In the Rubik's Cube, the effective branching factor is much larger (around 12–18), and plain DFS does not guarantee a shortest or even reasonably short solution. Since finding a near-optimal or at least reasonably short solution is important for this project, DFS is not suitable. Moreover, due to its depth-first nature, DFS follows a single path very deeply before backtracking. In a huge search space like the Rubik's Cube, this can result in exploring many extremely long and unproductive paths before even

getting close to the goal. As a result, the total search time can become impractically large, even though the memory usage of DFS is relatively low. For these reasons, I chose not to use DFS.

3. **Bidirectional BFS (Not selected)**
   Unlike BFS, Bidirectional BFS runs two BFS searches simultaneously. One from the start node(scrambled cube) and one from the target node(solved cube). When the two searches meet in the middle, it reduces the search space. Hence, the time complexity becomes O(b^d/2)+O(b^d/2), which is smaller than O(b^d). It looks like a more feasible alternative to standard BFS. However, for the Rubik's Cube, where the branching factor is approximately b=12 and the solution depth is around d=20, Bidirectional BFS still requires about 12^10 states for depth 10, which can cause OutOfMemoryError.



4. **A* (Selected:Failed)**
   A* uses an estimate of the distance from each state to the target. Each state vvv is evaluated using

$$f(v)=g(v)+h(v)$$

   where
   - g(v)=distance travelled from start to v
   - h(v)=estimated distance from v to target

   To ensure that A* algorithm finds the solved cube, I need this condition as the following:
   A* with admissible heuristics Theorem : if we use min-PriorityQueue and h(v) ≤ dist(v, target) for all v, then A* will eventually find the target.

   A* explore the most promising states first rather than exploring the entire states like BFS. Hence, this allows the search to be guided toward the solution rather than scanning the entire search space. However, A* still requires significant memory because it stores both the frontier (OpenSet) and all visited states (ClosedSet). Given the Rubik's Cube's branching factor of approximately 12 and solution depth around 20, A* can easily reach memory limits depending on the heuristic used. Despite this limitation, I tried this because I thought a good heuristic can make it feasible.

   4.1. Attempt1: Weighted A* with Misplaced Cubie Heuristic
   I implemented a SolverAstar class using a Priority Queue and a HashMap for visited states.

   - Heuristic Design: I used a simple admissible heuristic (cubieHeuristic) that counted the number of misplaced corners and edges. Since a single move effects 4 corners and 4

$$h(n) = \max\left(\left\lceil\frac{\text{Wrong Corners}}{4}\right\rceil, \left\lceil\frac{\text{Wrong Edges}}{4}\right\rceil\right)$$

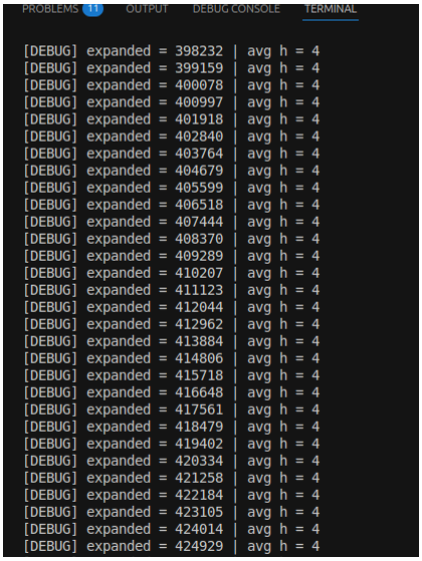
edges, the logic was:

- **Weighting:** To speed up the search, I applied a HEURISTIC_WEIGHT of 3.0, effectively turning it into Weighted A* (WA*).

4.1-1. Reason for Failure 1: Weak Heuristic & Branching Factor

The heuristic described above was **too weak** (it underestimated the true distance too much).

- For a cube that requires 10 moves to solve, this heuristic often returned a value of only 2 or 3.
- Because h was small, the total cost f(n) = g(n) + h(n) did not grow fast enough to distinguish good paths from bad ones.
- The A* algorithm behaved almost like a Breadth-First Search (BFS). With a branching factor of approx. 13, searching to depth 10 required expanding tens of billions of nodes, which was impossible within the 10-second limit.



4.1-2. Reason for Failure 2: Object Creation Overhead

The initial RubiksCube class represented the state using a 3D character array (char[][][] state).

- **Deep Copying:** In SolverAstar, every time a node was expanded (generating 12-18 neighbors), a new RubiksCube object was instantiated using a deep copy constructor.
- **Memory Exhaustion:** Creating millions of objects and 3D arrays rapidly filled the Java Heap Space, triggering frequent Garbage Collection and eventually causing an OutOfMemoryError.
- **Solution:** This failure led me to switch to the **Coordinate Vector** representation (using int arrays for Permutation/Orientation) in the final project, which requires no object creation during the search.

Result: Failed (OutOfMemoryError)

This led me to switch to coordinate representation.

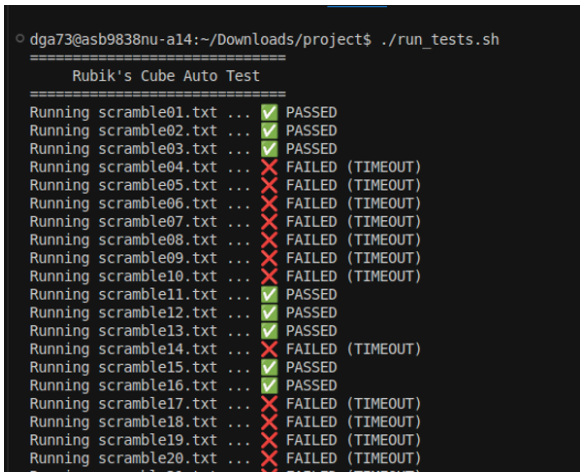
4.2. Attempt2: Weighted A with Object-Oriented State*

- **Heuristic:** A weighted admissible heuristic (h(n)* 3.0) based on the count of misplaced cubies
- **Pruning:** Implemented redundancy checks to skip move sequences like F F F or commutative pairs like F B vs B F.
- **Data Structures:** Used a PriorityQueue for the Open Set and a HashMap<Long, Integer> for the Closed Set (using a rolling hash of the state).
- **Result Analysis (9/40 Passed)**

- This solver successfully solved simple scrambles (short depth) but consistently failed on harder inputs due to **Time Limit Exceeded (TLE)** or **Memory Limit Exceeded (MLE)**. The 9 passed cases were likely scrambles requiring fewer than 10-12 moves

4.3. Attempt 3: 3D Manhattan Distance Heuristic*

- **Implementation:** I defined 3D coordinates for all 8 corners and 12 edges. The heuristic calculated $\sum (|x_1-x_2| + |y_1-y_2| + |z_1-z_2|)$ and divided it by a factor to keep it admissible/consistent.
- **Configuration:** Heuristic Weight set to 3.0. Redundancy pruning (FFFF loops) was partially disabled or adjusted.



The result was **8 passed out of 40**, which was slightly worse than the simple "misplaced count" heuristic

4.4. Attempt4: : Greedy Best-First Search with Manhattan Heuristic

Following the initial A* attempt, I tried to make the search more aggressive (Greedy) to speed up the process.

- **Modifications:**
  - **Heuristic Weight:** Increased HEURISTIC_WEIGHT to **5.0**. This effectively transformed the algorithm from A* into a Greedy Best-First Search.
  - **Heuristic Function:** Switched to a manhattanHeuristic() (sum of distances of stickers/cubies to their solved faces) instead of the simple mismatch count.
  - **State Representation:** Still relied on the object-oriented RubiksCube class with deep copying.
- **Result Analysis (6/40 Passed)**
- Performance **degraded** compared to the previous attempt (dropping from 9 to 6 solved cases)

4.5. Attempt 5: Optimized Weighted A with Pruning (Best A Result)

- **Algorithm Details:**
  - **Heuristic:** Misplaced Cubie Count (cubieHeuristic) with a weight of **3.0**. This provided a balance between speed and solution quality.
  - **Pruning:** I implemented **Commutative Pruning** (e.g., preventing B F if F B was already checked) and **Loop Pruning** (preventing F F F if it leads back to the start). This reduced the effective branching factor from ~18 to ~13.3.
  - **Data Structure:** PriorityQueue for the frontier and HashMap for visited states

- **Result Analysis (10/40 Passed)**
    - This version achieved the highest success rate among the non-Kociemba approaches, solving **10 out of 40 test cases**

4.5-1. Technical Causes of Failure (Despite Optimizations)

1. **Memory Explosion (The "Closed Set" Problem):**
    - Even with the reduced branching factor, storing visited states in a HashMap is memory-intensive. At depth 15, the number of unique states exceeds tens of millions.
    - The JVM's Heap Space (even with 2GB+) filled up rapidly, causing the program to crash or stall due to excessive Garbage Collection.
2. **Object Overhead:**
    - The solver relied on new RubiksCube(current.cube) for every move. Creating millions of objects overwhelmed the memory allocator, confirming that **Java's object overhead is too high** for high-performance combinatorial search without using primitive data types (like int arrays or bitboards).

4.6. Conclusion from Failure

This experiment proved that heavy object manipulation and weak heuristics are non-viable for the Rubik's Cube. This led to the decision to switch to IDA* (to save memory) and Kociemba's Two-Phase Algorithm (to solve deeper states efficiently using pre-computed integer tables).

5. **IDA* (Iterative Deepening A*) (Selected)**
   IDA* (Iterative Deepening A*) is more efficient than A* algorithm in terms of memory usage.
   A* has to store a large PriorityQueue (the open set) and a large HashSet (the closed set).
   Because, in rubik's cube the number of explored states grows exponentially.
   IDA* = A* + Depth-First Search. It keeps the strong cost function from A* and explores states using DFS-style. This gives IDA* two major advantages: very low memory usage (comparable to DFS) more efficient than A* algorithm in terms of memory usage.
   A* has to store a large PriorityQueue (the open set) and a large HashSet (the closed set).
   Because, in rubik's cube the number of explored states grows exponentially.
   IDA* alone is not enough for Rubik's Cube, but when combined with **Pattern Databases (PDB)**, it becomes extremely powerful.

6. **Final Selection (Kociemba's Two-Phase Algorithm)(Solved 40/40)**
   Resources
    - **https://kociemba.org/cube.htm**
    - https://medium.com/@muhammadalikhan0003/kociembas-two-phase-algorithm-how-it-works-and-its-applications-3d8f97a3562a
    - https://russfeld.me/projects/rubiks/presentation.pdf

   For this project, I chose to implement **Kociemba's Two-Phase Algorithm** combined with *IDA (Iterative Deepening A)*** search.

6.1 Data Structures and Class Structures

- RubiksCube.java (Input/Output): Handles file parsing and primitive representation (Facelet based). It converts the input string into the internal CubieCube structure (Reads input→Builds 54 facelets →Converts to CubieCube)

- CubieCube.java (State Representation): Represents the cube using Coordinate (Corner Permutation, Corner Orientation, Edge Permutation, Edge Orientation): I used integer arrays (cp, co, ep, eo) instead of 2D arrays or objects for performance speed during the search
- Phase1.java & Phase2.java (Solver Logic): These classes contain the pre-computed Move Tables (transition matrices) and Pruning Tables (heuristics).
    - *Optimization:* Instead of calculating the result of a move at runtime, I pre-computed transition tables (EO_MOVE[2048][18], CO_MOVE[2187][18],SLICE_MOVE[495][18],CP_MOVE, UDEdge_MOVE, SlicePerm_MOVE for Phase 2) so that applying a move is just an O(1) array lookup
    - Phase 1:Edge Orientation, Corner Orientation, Middle slice edges placement and its goal reduces search space from 4.3×10^{19} to 19 million.
    - Phase2: Corner permutation,Edge permutation, Slice permutation, Last layer alignment

- Solver.java : Serves as the entry point of the program. It implements the *IDA (Iterative Deepening A*)** algorithm logic. It orchestrates the solving process by executing the Phase 1 search loop first, and upon finding a partial solution, transitions the state to Phase 2 to complete the solution. It also manages the global moveStack and handles the final output generation.

6.2 Heuristics

- Instead of traditional heuristics like Manhattan Distance (which is too weak for Rubik's Cube), I implemented Pattern Databases (Pruning Tables).
    - **Phase 1 Heuristic:** max(Slice_EO_Prune, Slice_CO_Prune)
      I calculate how many moves it takes to solve *just* the edge orientations (EO) and *just* the corner orientations (CO) combined with the slice edges. We take the maximum of these values as an admissible heuristic.
    - **Phase 2 Heuristic:** max(CP_UD_Prune, SlicePerm_Prune)
      Similarly, I look up the pre-calculated distance for corner permutations and edge permutations relevant to Phase 2.
      Was one heuristic better? The combination of Slice_EO and Slice_CO was significantly better than using just one. Using only one resulted in the search tree expanding too fast. The max() function effectively "prunes" branches that are theoretically impossible to solve within the current depth limit.

6.3. Details

The most crucial implementation step involves the abstraction of the physical cube state into a computationally viable model. This requires translating the facelet representation read from the input file into the internal **Cubie Coordinates** used by the search algorithms.
The following arrays serve as the **Facelet-to-Cubie Mapping Layer**, which is the foundational bridge for state conversion performed by the RubiksCube class:

```
private static final int[][] CORNER_STICKERS = {
    {8, 9, 20},   // URF
    {6, 18, 38},  // UFL
    {0, 36, 47},  // ULB
    {2, 45, 11},  // UBR
    {29, 26, 15}, // DFR
    {27, 44, 24}, // DLF
    {33, 53, 42}, // DBL
    {35, 17, 51}  // DRB
};
```

```
private static final int[][] EDGE_STICKERS = {
    {5, 10},  // UR
    {7, 19},  // UF
    {3, 37},  // UL
    {1, 46},  // UB
    {23, 12}, // FR
    {21, 41}, // FL
    {50, 39}, // BL
    {48, 14}, // BR
    {32, 16}, // DR
    {28, 25}, // DF
    {30, 43}, // DL
    {34, 52}  // DB
};
```

- **Mapping:** The arrays define the exact location of the 54 individual sticker positions (Facelets) relative to the 20 internal pieces (Cubies).
- **Derivation:** The indices within these arrays were manually derived and hardcoded based on the solved state's color mapping. This allows for rapid, constant-time O(1) determination of any cubie's position and orientation upon initialization or verification, eliminating the need for complex runtime calculations.
- **Cubie Movement** is really important because if you map it wrong, it might produce a long incorrect solution, which I had previously with incorrect mapping.
- **These parts were the most time-consuming because every sticker index had to be hard-coded by determining exactly which position each color belongs to. After completing the mapping, my solver successfully solved all 40/40 test cases, and I verified each result using RubiksCube.is_solved(). I previously obtained incorrect solutions, and fixing the mapping fully resolved this issue.**

```
public void moveU() {

    int tcp = cp[3]; cp[3] = cp[2]; cp[2] = cp[1]; cp[1] = cp[0]; cp[0] = tcp;
    // CO: No change
    int tco = co[3]; co[3] = co[2]; co[2] = co[1]; co[1] = co[0]; co[0] = tco;
    // EP: 0 -> 1 -> 2 -> 3 -> 0
    int tep = ep[3]; ep[3] = ep[2]; ep[2] = ep[1]; ep[1] = ep[0]; ep[0] = tep;
    // EO: No change
    int teo = eo[3]; eo[3] = eo[2]; eo[2] = eo[1]; eo[1] = eo[0]; eo[0] = teo;
```